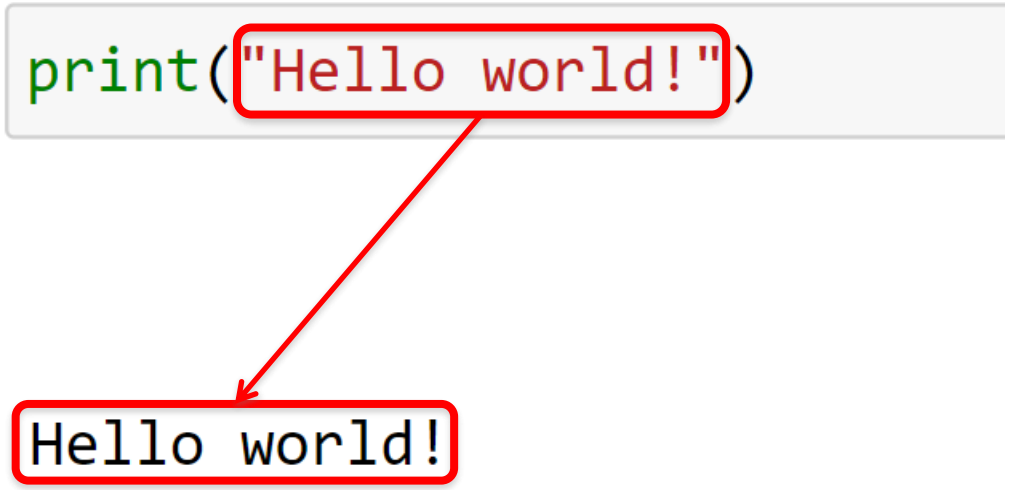
```
print("Computer Programming 2026\n")
print("Department of Geography\n")
print("12 March 2026\n")

print("Week 3")
```

The first program: Hello world!



- Print out the message "**Hello world!**" on screen
- print is a function
- The message to print is included between ""

Instructions and instruction blocks

- In a sequence of *instructions,* every instruction ends with the “return/enter key”
- To indicate an instruction block, you must indent (using the “tab key”) each line of the block by the same amount

Structure of a program

- **Declarative part :** includes the declarations of the elements of the program
  - Libraries and functions

- **Executive part :** includes the instructions to execute, which falls under the categories:
  - Assignment instructions (=)
  - Control instructions:
    - Conditional (if-else, …)
    - Iterations, or **loops** (while, for, …)
  - Input/Output instructions (print, input, ...)

Functions

Functions are subprograms, and can be of two different types

- *Functional* type
- *Procedural* type

- Fuctional-type subprograms can be considered as an abstraction of value
  - calling the function means to associate to the name of the function the value representing the calculated result of the subprogram

```
result = sum(A,B)
```

- Procedural-type subprograms can be considered as an abstraction of operations
  - calling the function means to execute the instructions of the subprogram, which perform the operation specified by the subprogram

```
read(A,B)
```

Functions

Using a function implies:

1. Defining the function
2. Calling the function (wherever the function is needed in the code)

FOR NOW… WE ONLY CALL!

Calling a function

Within your program, it indicates the location where the code inside the function definition must be executed.

- Calling a function:
  - as an operation implying a “visible” outcome:

```
outcome = functionname(parameter1, …)
```

  - as an instruction:

```
functionname(parameter1, …)
```

- `parameter1, …` are the parameter values that must match, in terms of order and type, the formal parameters of the function (as defined in the definition step)
- The “parameter values” can be variables, constants, or expressions, whose values are copied in the formal parameters of the function at the moment you call the function

Print

- print is used to print out the output of the program on screen
- print is a function
- The message to print is included between “”

Let’s go back to the GCD pseudocode

1. **Read** A and B
2. min = the minimum between A and B
3. found = 0, GCD = min
4. **As long as** found != 1
    1. **If** GCD divides A and B
        1. **Then** found = 1
        2. **Else** GCD = GCD - 1
5. **Print** GCD

Print out on screen the message “**Insert the first value: ”**

```
print(“Insert the first value: ”)
```

How do we interact with the "*outside*"?

Output

- print is used to print out the output of the program on screen

Input

- input is used to provide inputs, e.g. from the keyboard, to our program

Let’s go back to the GCD pseudocode

1. **Read** A and B
2. min = the minimum between A and B
3. found = 0, GCD = min
4. **As long as** found != 1
    1. **If** GCD divides A and B
        1. **Then** found = 1
        2. **Else** GCD = GCD - 1
5. **Print** GCD

```
A = input(“Insert the first value: ”)
B = input(“Insert the second value: ”)
```

but… A and B???

Variables

```
A = input(“Insert the first value: ”)
```

Variables are containers (memory areas) identified by a **univocal** name

Variables are defined by a **type** and a **name**

Variables

```
A = input("Insert the first value: ")
```

Which meaning would we give to the type of this input?

Integer numbers

Variables

```
A = input("Insert the first value: ")
```

Which meaning would we give to the type of this input?

```
type(A)
```

```
str
```

Integer numbers

The acquired input from keyboard must be transformed into an integer

```
type(int(A))
```

```
int
```

```
A = int(input("Insert the first value: "))
```

Let’s go back to the GCD pseudocode

1. **Read** A and B
2. min = the minimum between A and B
3. found = 0, GCD = min
4. **As long as** found != 1
    1. **If** GCD divides A and B
        1. **Then** found = 1
        2. **Else** GCD = GCD - 1
5. **Print** GCD

```
A = int(input("Insert the first value: "))
B = int(input("Insert the second value: "))
```

Exercise

- Problem
  - Write a program that asks for inserting a character and then prints it out on screen
- Sub-problem
  - How do I save the character?
  - How do I print the character?

Exercise

- Problem
  - Write a program that asks for inserting a character and then prints it out on screen

```
character = input("Insert a character: ")
print("The inserted character is:", character)
```

Execution of the assignments

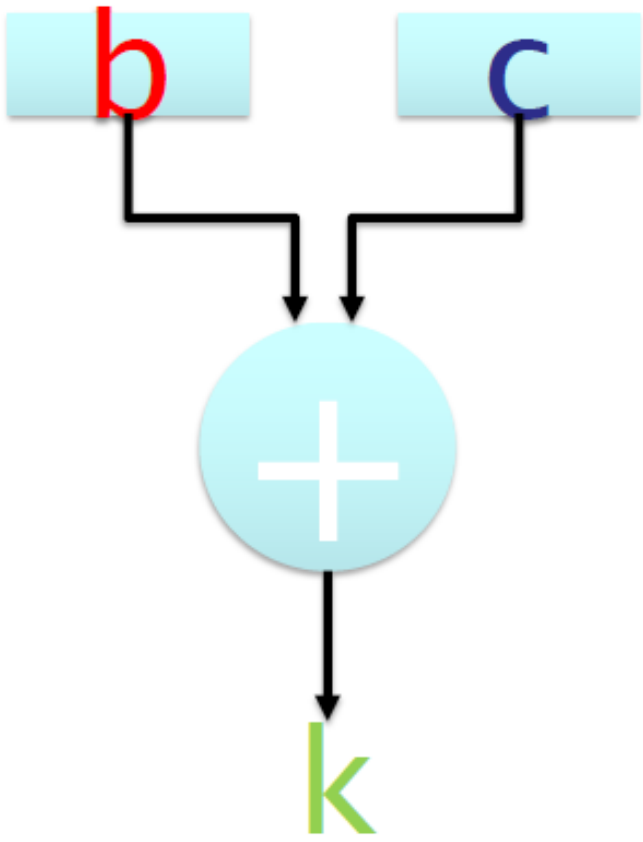
- **evaluation of the <u>expression</u>** that appears to the *right* of the symbol =
  - the value of the variables in such expression is read from the corresponding memory cells
- **saving the result** of the expression in the **<u>variable</u>** to the *left* of the symbol =

Execution of the assignments

$$k = b + c$$

Execution of the assignments

$k = b + c$

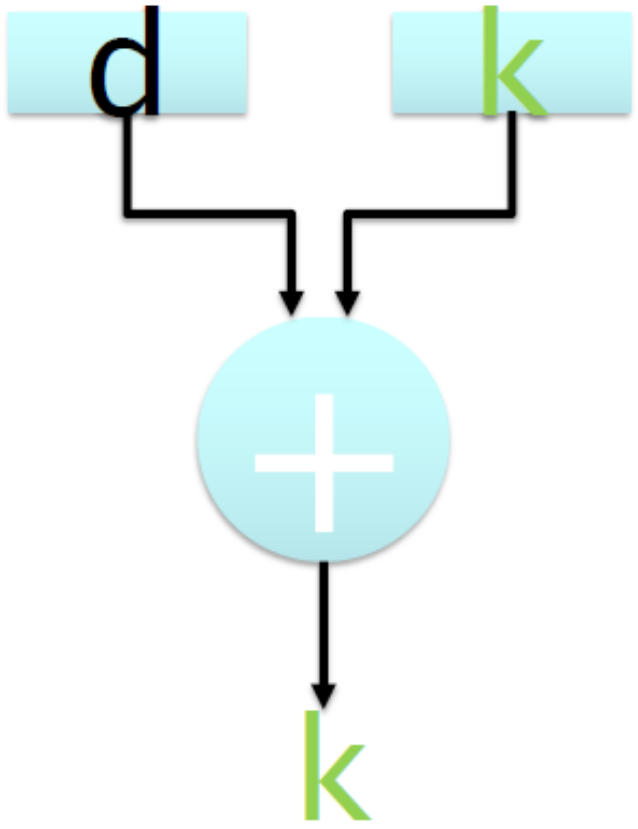


Execution of the assignments

$$k = k + d$$

Execution of the assignments

$k = k + d$



Examples of assignment

```
x = 23
w = "a"
y = z
alfa = x + y
r3 = (alfa * 43 - xgg) * (delta - 32 * j)
x = x + 1
```

Instructions in the form of ***variable = variable operator expression*** can be written as: ***variable operator= expression***

Abbreviations (*operators* of assignment):

```
a = a + 7     a = a * 5     a = a - 1     a = a / 2
a += 7        a *= 5        a -= 1        a /= 2
```

Arithmetic

- Arithmetic operators in Python:
  - `*` for multiplication and `/` for division
  - The *module* operator calculates the **remainder** of the division:

    `13 % 5` is equal to `3`

- Priority of operators:
  - As in arithmetic, multiplication and division have priority over sum and subtraction
  - use the brackets when there is ambiguity
    - For example: the arithmetic mean of **a**, **b**, **c**:

```
 a + b + c / 3          NO !!!!
(a + b + c) / 3         YES
```

Arithmetic

Operation	Python operator	Arithmetic expression	Python expression
Sum	`+`	$f+7$	`f + 7`
Subtraction	`-`	$p-c$	`p - c`
Multiplication	`*`	bm	`b * m`
Division	`/`	x/y	`x / y`
Module	`%`	$r \bmod s$	`r % s`

Python operators	Operations	Priority
`( )`	Brackets	Evaluated as the first ones. If there is any nesting, the most internal brackets are evaluated first. If there are multiple brackets at the same level, they are evaluated from left to right.
`* , / , %`	Multiplication, Division, Module	Evaluated as the second ones. If there are multiple of such operators, they are evaluated from left to right.
`+ , -`	Sum, Subtraction	Evaluated as the last ones. If there are multiple of such operators, they are evaluated from left to right.

Let's continue with our GCD...

Let’s go back to the GCD pseudocode

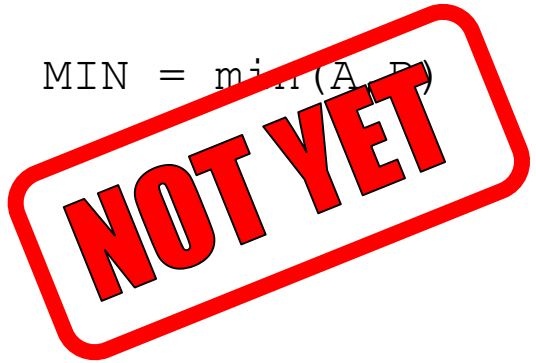
1. **Read** A and B
2. min = the minimum between A and B
3. found = 0, GCD = min
4. **As long as** found != 1
    1. **If** GCD divides A and B
        1. **Then** found = 1
        2. **Else** GCD = GCD - 1
5. **Print** GCD

```
A = int(input(“Insert the first value: ”))
B = int(input(“Insert the second value: ”))
```

Let’s go back to the GCD pseudocode

1. **Read** A and B
2. min = the minimum between A and B
3. found = 0, GCD = min
4. **As long as** found != 1
    1. **If** GCD divides A and B
        1. **Then** found = 1
        2. **Else** GCD = GCD - 1
5. **Print** GCD

```
A = int(input("Insert the first value: "))
B = int(input("Insert the second value: "))

MIN = min(A,B)
```



Let’s go back to the GCD pseudocode

1. **Read** A and B
2. min = the minimum between A and B
3. found = 0, GCD = min
4. **As long as** found != 1
    1. **If** GCD divides A and B
        1. **Then** found = 1
        2. **Else** GCD = GCD - 1
5. **Print** GCD

```
A = int(input("Insert the first value: "))
B = int(input("Insert the second value: "))
```

```
if A < B:
    MIN = A
else:
    MIN = B
```

is A lower than B?

- **True**: MIN is A
- **False**: MIN is B

Conditional execution

- If a condition is satisfied, it assumes a value equal to True or 1
- If a condition is not satisfied, it assumes a value equal to False or 0

Exercise

Write a program that, given a number, says if this number is positive or not.

Solution

1. Insert N
2. is N higher than 0?

   True: N is positive

   False: N is not positive

Exercise

```
n = int(input("Insert a number: "))

if n > 0:
    print("\nA positive number!\n")
else:
    print("\nA negative number (or zero)\n")

print("End of the program")
```

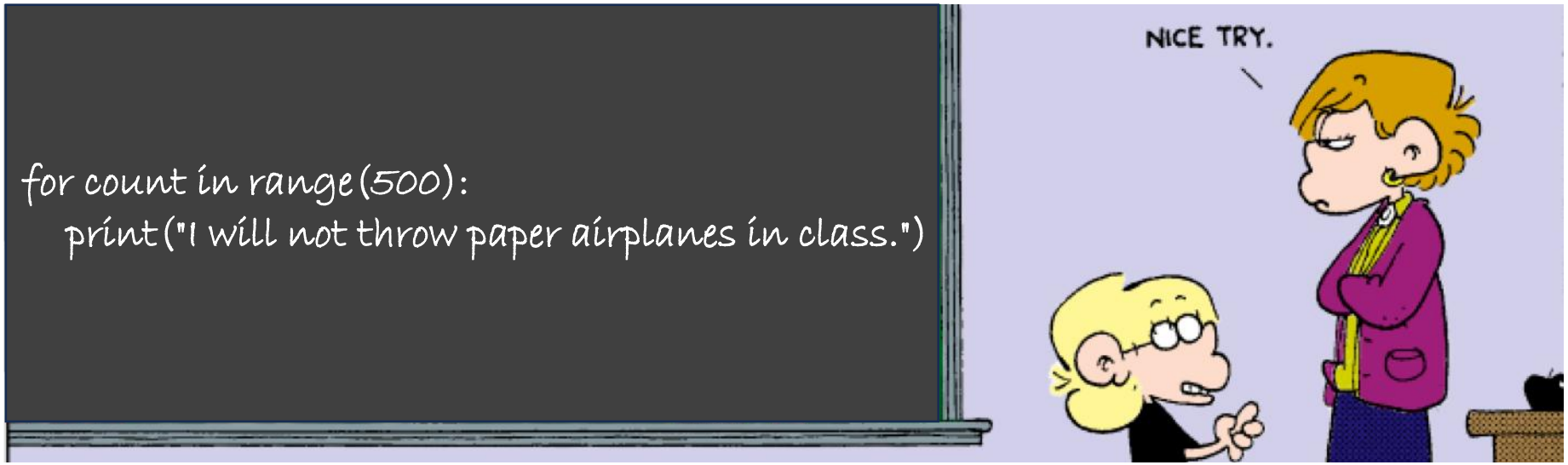
Let’s go back to the GCD pseudocode

1. **Read** A and B
2. min = the minimum between A and B
3. found = 0, GCD = min
4. **As long as** found != 1
    1. **If** GCD divides A and B
        1. **Then** found = 1
        2. **Else** GCD = GCD - 1
5. **Print** GCD

```
A = int(input("Insert the first value: "))
B = int(input("Insert the second value: "))

if A < B:
    MIN = A
else:
    MIN = B

found = 0
GCD = MIN
```

Objectives

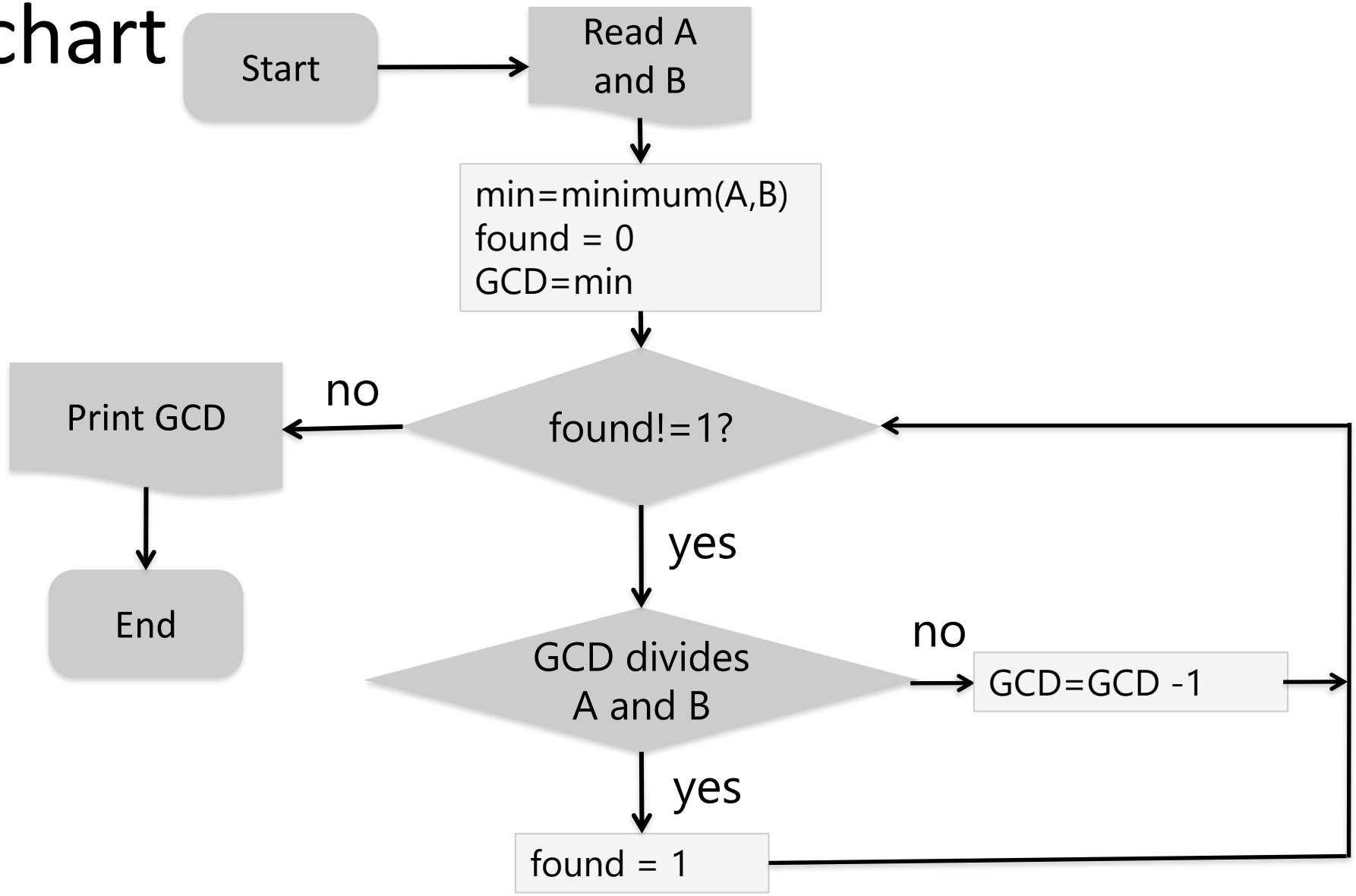
- Iterations
  - while
  - for



GCD: pseudocode

1. **Read** A and B
2. min = the minimum between A and B
3. found = 0, GCD = min
4. **As long as** found != 1
    1. **If** GCD divides A and B
        1. **Then** found = 1
        2. **Else** GCD = GCD - 1
5. **Print** GCD

GCD: flowchart



Let’s go back to the GCD pseudocode

1. **Read** A and B
2. min = the minimum between A and B
3. found = 0, GCD = min
4. **As long as** found != 1
    1. **If** GCD divides A and B
        1. **Then** found = 1
        2. **Else** GCD = GCD - 1
5. **Print** GCD

```
A = int(input("Insert the first value: "))
B = int(input("Insert the second value: "))

if A < B:
    MIN = A
else:
    MIN = B

found = 0
GCD = MIN

...

...

print("Greatest Common Divisor =", GCD)
```

Let’s go back to the GCD pseudocode

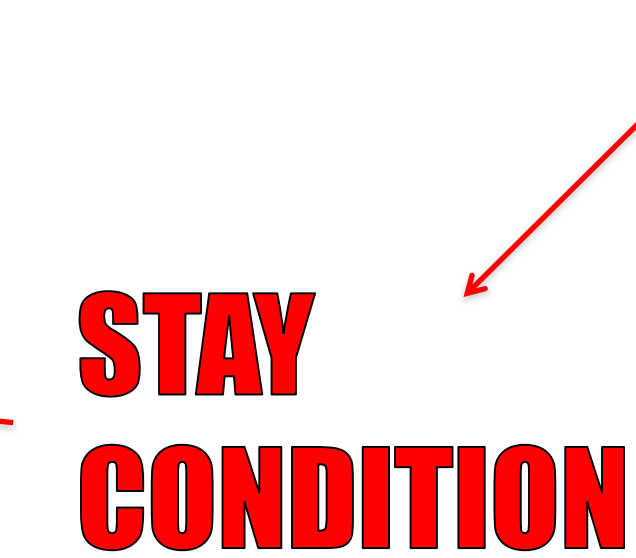
1. **Read** A and B
2. min = the minimum between A and B
3. found = 0, GCD = min
4. **As long as** found != 1
    1. **If** GCD divides A and B
        1. **Then** found = 1
        2. **Else** GCD = GCD - 1
5. **Print** GCD

```
A = int(input("Insert the first value: "))
B = int(input("Insert the second value: "))

if A < B:
    MIN = A
else:
    MIN = B

found = 0
GCD = MIN

while found != 1:
    ...
    ...

print("Greatest Common Divisor =", GCD)
```

WHILE

Iterates the execution of **an instruction as long as** a certain condition is TRUE

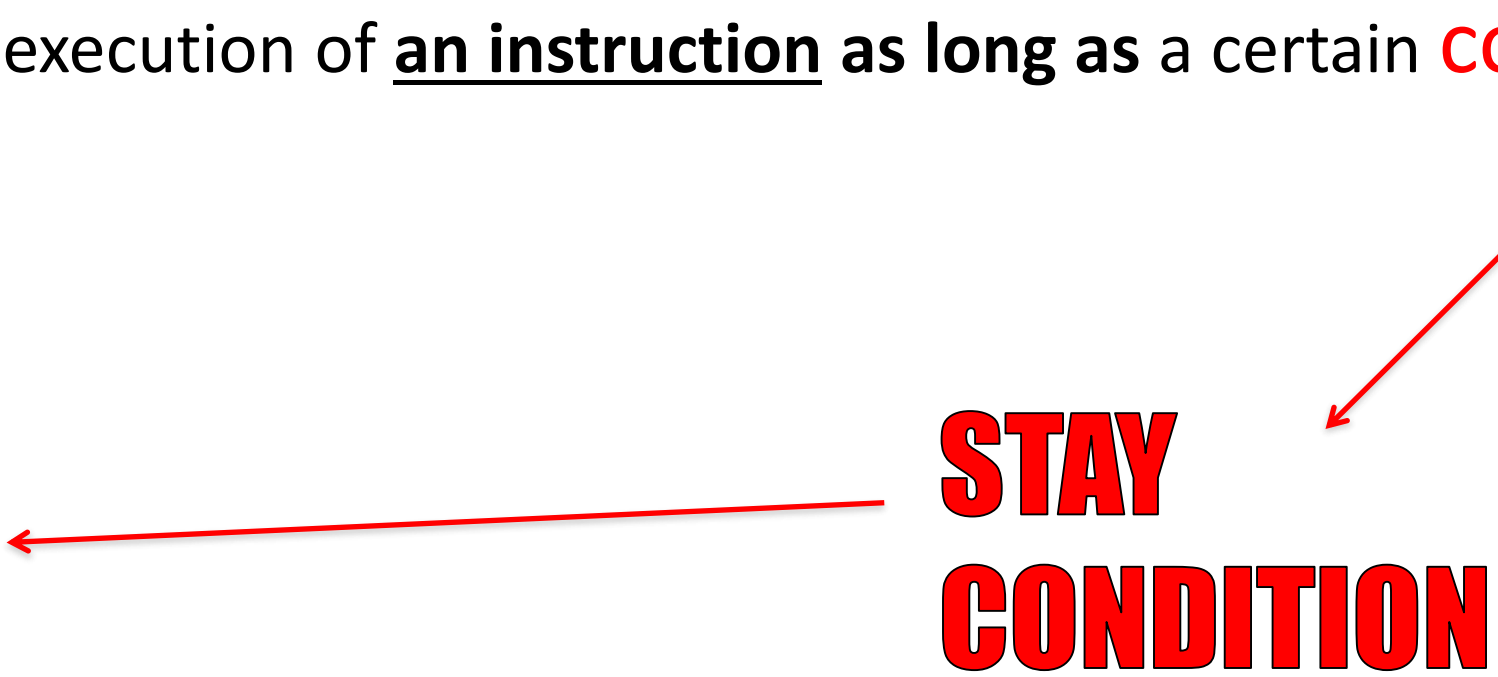
```
found = 0
while found != 1:
    …
    do something ☺
    …
```



WHILE (example)

Iterates the execution of **an instruction as long as** a certain condition is TRUE

```
a = 5
b = 4

while b > 0:
    a += a
    b -= 1

print("The value of a is now:", a)
```



Let's go back to the GCD pseudocode

1. **Read** A and B
2. min = the minimum between A and B
3. found = 0, GCD = min
4. **As long as** found != 1
    1. **If** GCD divides A and B
        1. **Then** found = 1
        2. **Else** GCD = GCD - 1
5. **Print** GCD

```
A = int(input("Insert the first value: "))
B = int(input("Insert the second value: "))

if A < B:
    MIN = A
else:
    MIN = B

found = 0
GCD = MIN

while found != 1:
    if A % GCD == 0 and B % GCD == 0:
        found = 1
    else:
        GCD = GCD - 1

print("Greatest Common Divisor =", GCD)
```

REVIEW ON ALGEBRA AND LOGIC...

Logical fundamental operations

- Binary logical operators (with 2 logical operands)
  - **OR** operator, or *logical sum*
  - **AND** operator, or *logical product*
- Unary logical operator (with 1 logical operand)
  - **NOT** operator, or *negation*, or *inversion*

As the logical operands allow only two values, we can completely define every logical operator through a ***table*** that associates operands and results

Logical fundamental operations

The *variables* of boolean algebra can assume only the two values 0 and 1

- precisely, if `x` represents a variable, we have:
  - `x = 0` if and only if `x ≠ 1`
  - `x = 1` if and only if `x ≠ 0`

- `+(OR)` and `*(AND)` are defined as:

+	0	1
0	0	1
1	1	1

*	0	1
0	0	0
1	0	1

- The *negation* (`NOT`) is defined as:

!	
0	1
1	0

Logical fundamental operations

A	B	A or B
0	0	0
0	1	1
1	0	1
1	1	1

(logical sum)

A	B	A and B
0	0	0
0	1	0
1	0	0
1	1	1

(logical product)

A	not A
0	1
1	0

(negation)

The tables list all possible input combinations and the corresponding result of each combination

Back to our GCD...

Let’s go back to the GCD code

1. **Read** A and B
2. min = the minimum between A and B
3. found = 0, GCD = min
4. **As long as** found != 1
    1. **If** GCD divides A and B
        1. **Then** found = 1
        2. **Else** GCD = GCD - 1
5. **Print** GCD

```
A = int(input("Insert the first value: "))
B = int(input("Insert the second value: "))

if A < B:
    MIN = A
else:
    MIN = B

found = 0
GCD = MIN

while found != 1:
    if A % GCD == 0 and B % GCD == 0:
        found = 1
    else:
        GCD = GCD - 1

print("Greatest Common Divisor =", GCD)
```

GCD: zoom

```
while found == 1:
    if A % GCD == 0 and B % GCD == 0:
        found = 1
    else:
        GCD = GCD - 1
```

P1

P2

GCD: conditional statement

- P1: `if A % GCD == 0`
- P2: `if B % GCD == 0`

P1	P2
0	0
0	1
1	0
1	1

GCD: conditional statement

- P1: `if A % GCD == 0`
- P2: `if B % GCD == 0`

P1	P2	
0	0	0
0	1	
1	0	
1	1	

GCD: conditional statement

- P1: `if A % GCD == 0`
- P2: `if B % GCD == 0`

P1	P2	
0	0	0
0	1	0
1	0	
1	1	

GCD: conditional statement

- P1: `if A % GCD == 0`
- P2: `if B % GCD == 0`

P1	P2	
0	0	0
0	1	0
1	0	0
1	1	

GCD: conditional statement

- P1: `if A % GCD == 0`
- P2: `if B % GCD == 0`

P1	P2	
0	0	0
0	1	0
1	0	0
1	1	1

DO YOU REMEMBER?

Logical fundamental operations

A	B	A or B
0	0	0
0	1	1
1	0	1
1	1	1

(logical sum)

A	B	A and B
0	0	0
0	1	0
1	0	0
1	1	1

(logical product)

A	not A
0	1
1	0

(negation)

The tables list all possible input combinations and the corresponding result of each combination

GCD: conditional statement

- P1: `if A % GCD == 0`
- P2: `if B % GCD == 0`

P1	P2	P1 AND P2
0	0	0
0	1	0
1	0	0
1	1	1

Exercise

Problem

- Find the largest number among N positive numbers that are typed on keyboard

Solution

- Know N
- Ask for inserting the N values
- Search for the largest among the N values

Exercise

```
N = int(input("How many numbers do you want to insert? "))

biggest = 0
count = 0
while count < N:
    num = int(input("Insert the new number (positive): "))
    if num > biggest:
        biggest = num
    count += 1

print("\nThe biggest number is", biggest)
```

See you
next week!